

# xxxx 实验报告

姓名： 林宏宇 学号： 23320093

## 1. 实验内容

### 2.1 ALU 设计实验

下图给出了一个具有 N 位输入和 N 位输出的算数逻辑单元的电路符号。算术逻辑单元接收说明执行哪个功能的控制信号 F，执行对应功能后输出 N 位结果。

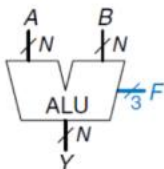


图 1

实验要求实现以下算术运算功能，其对应的指令码及功能如下：

表 1

| F <sub>2:0</sub> | 功能              | F <sub>2:0</sub> | 功能        |
|------------------|-----------------|------------------|-----------|
| 000              | A + B(Unsigned) | 100              | $\bar{A}$ |
| 001              | A - B           | 101              | SLT       |
| 010              | A AND B         | 110              | 未使用       |
| 011              | A OR B          | 111              | 未使用       |

本次实验将 ALU 输出结果通过板载串口进行显示验证，原理图如下图所示：

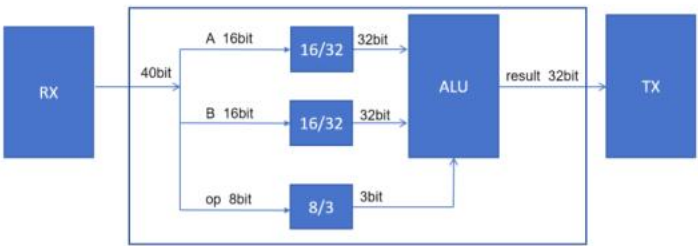


图 2

## 四、 实验要求

1. 根据 ALU 原理图(图 1)，使用 Verilog 语言定义 ALU 模块，其中输入输出端口参考实验原理，运算指令码长度为[2:0]。
2. 将串口收到的 40bit 数拆分为两个 16bit 数据与一个 8bit 数据，作为待计算数据 A、B 与操作符 op；
3. 将待计算数据 A、B 经过符号扩展至 32 位后，输入到运算器的端口 A、B；
4. 运算器支持“加、减、与、或”4 种运算，需要 3 位（8 个操作）。将操作符 op 截取低三位输入运算器的 op 端口作为运算器的控制信号(默认为高电平)；
- 5.实现 SLT 功能，ALU 功能实现 Z, N, C, V 功能
- 6.将计算 32 位结果 s 输出到串口显示。
- 7.验证表 1 中所有功能。
- 8.给出 RTL 源程序（.v 文件）

## 2. 实验过程

### 1. 添加 ip 核

```
>  u_convert_fifo : fifo_32b_2_8b (fifo_32b_2_8b.xci)
```

```
>  u_op_convert_fifo : fifo_8b_2_32b (fifo_8b_2_32b.xci)
```

### 2. 实现 alu\_top 顶层模块里的例化

```
// your ALU instantiation here  
alu u_alu(  
    .A_i      (data_A),  
    .B_i      (data_B),  
    .op_i     (op),  
    .result_o (alu_result)  
);  
// your ALU instantiation end
```

### 3. 实现 alu.v 模块的代码

- ① 根据 op\_i 的值判断进行什么运算
- ② 对 A\_i 和 B\_i 进行运算，将结果存到 result\_o
- ③ 同时要判断 Z N C V 的结果

```

always @(*) begin
    // Initialize outputs
    result_o = 32'b0;
    Z = 1'b0;
    N = 1'b0;
    C = 1'b0;
    V = 1'b0;

    case (op_i)
        3'b000: begin // Addition
            {C, result_o} = A_i + B_i; // Capture carry out
            V = (A_i[31] & B_i[31] & ~result_o[31]) | (~A_i[31] & ~B_i[31] & result_o[31]); // Overflow
        end
        3'b001: begin // Subtraction
            {C, result_o} = A_i - B_i; // Capture carry out
            V = (A_i[31] & ~B_i[31] & ~result_o[31]) | (~A_i[31] & B_i[31] & result_o[31]); // Overflow
        end
        3'b010: begin // AND
            result_o = A_i & B_i;
        end
        3'b011: begin // OR
            result_o = A_i | B_i;
        end
        3'b101: begin // ~
            result_o = A_i | B_i;
        end
        3'b101: begin // SLT (Set Less Than)
            result_o = (A_i < B_i) ? 32'b1 : 32'b0;
        end
        default: begin // Default case: do nothing
            result_o = 32'b0;
        end
    endcase

    // Set flags
    Z = (result_o == 32'b0); // Zero flag
    N = result_o[31]; // Negative flag
end

```

## 4. 结果分析

ATK

EDM

正点原子串口调试助手 V2.8

[2024-11-22 21:42:32.972]

TX: 0012003400

[2024-11-22 21:42:34.003]

RX: 00 00 00 46

[2024-11-22 21:42:45.761]

TX: 0012003401

[2024-11-22 21:42:46.786]

RX: FF FF FF DE

[2024-11-22 21:42:50.003]

TX: 0012003402

[2024-11-22 21:42:51.018]

RX: 00 00 00 10

[2024-11-22 21:42:55.557]

TX: 0012003403

[2024-11-22 21:42:56.577]

RX: 00 00 00 36

自行测试结果与实验要求一致，验收通过，程序正确

## 5. 总结

- 1.这一次的实验相对简单，顶层模块中已经实现好了将输入拆分的译码阶段等等，只需要读懂代码，在 alu.v 中接入顶层模块的寄存器，根据要求进行计算即可
- 2.实验操作流程也逐渐熟悉起来了，添加 ip 核、创建模板、写代码、烧录、串口实验，已经能够逐渐掌握实验的操作